



Assessing the Potential Impact of Memory-Safe Languages for System Software Security

Sponsor: NSA, Neal Ziring
By: University of Arizona and UT Dallas Team 3



Agenda

- Purpose Statement
- Motivation
- Characterization of Methods Used
- Procedures/Plans
- Schedule
- Project Deliverables
- Limitations and Delimitations
- Detailed Findings
- Issues/Challenges
- Conclusions and Recommendations
- Investigator Biographies
- Bibliography
- Acronyms
- Terms/Definitions



Purpose Statement

- Assessment of traditional C/C++ memory vulnerabilities in C/C++ and WebAssembly.
- Exploration of memory safety features of Rust and Go.
- Identification of vulnerabilities through CWE analysis.
- Provision of coding examples in multiple languages.
- Testing in WebAssembly for superior memory safety with Rust and Go.
- Integration of AI tools like ChatGPT for effectiveness in vulnerability detection and remediation in source code.



Motivation

- Recognition of Growing Security Concerns
- Deficiencies of Traditional Languages
- Shift Towards Memory-Safe Languages
- WebAssembly's Role
- Innovation Through AI Tools



Characterization of Methods Used

- Selection of Common Weakness Enumerations (CWEs)
- Development of Test Cases
- Analysis of WebAssembly Vulnerabilities
- Migration Strategies and Mitigation Techniques
- Integration and Testing of AI Tools
- Comparative Analysis



Procedures/ Plans

- Research into memory Common Weakness Enumerations (CWEs) in WebAssembly.
- Division of tasks among team members into two groups: programming and research.
- Creation of C/C++ examples to exploit vulnerabilities in both native environments and WebAssembly.
- Comprehensive study of WebAssembly-related cases in the National Vulnerability Database (NVD)
- Implementation of traditional migration strategies in native environments, coupled with a literature review.
- Testing the feasibility of traditional migration strategies in the WebAssembly environment.
- Conclusion of the literature review.
- Recreation of C/C++ code using Rust and Go to validate AI tools' effectiveness in detecting and remediating vulnerabilities.
- Testing Rust and Go code behavior in the WebAssembly environment.
- Statistical analysis of results from different AI tools to understand their effectiveness in securing the system.



Schedule

January 23, 2024	Project Bids
January 26, 2024	Project assignments
February 4, 2024	Project proposal
February 5 - February 12, 2024	Research memory CWE's that happen in WebAssembly
February 13 - February 26, 2024	Implement C/C++ examples exploiting in native environment and WebAssembly
February 26 - March 11, 2024	Implement traditional migration in native environment
March 11 - March 21, 2024	Testing traditional migration availability in WebAssembly
March 10- March 21, 2024	Midterm report writing and presentation practice run
March 22, 2024	Midterm presentations / Midterm deliverables
March 23 - April 8, 2024	Recreate C/C++ code in Rust and Go
April 9 - April 25, 2024	Testing Rust and Go code behavior in WebAssembly
April 26, 2024	Final report writing and presentation practice run
May 3, 2024	Final presentations / Project deliverables



Project Deliverables

- A comprehensive final report documenting research methodologies, findings, and insights.
- Professionally crafted presentation materials for academic and industry audiences.
- Establishment of a dedicated repository for transparency and reproducibility.
- A curated collection of test cases in the repository for replicating experiments.
- Analysis tools developed during the project housed in the repository.
- Aim to empower scholars, developers, and cybersecurity practitioners with resources to validate and build upon project outcomes.



Limitations and Delimitations

Limitations

- Scope of Programming Languages
- Depth of Vulnerability Analysis
- Testing Environment Constraints
- AI Tool Effectiveness

Delimitations

- Focused Study on Memory Safety
- Selection of WebAssembly as a Case Study
- Choice of Memory-Safe Languages
- Use of Specific AI Tools
- Empirical and Theoretical Approach



Detailed Findings

Review of memory-safe research in C/C++

	Native Environment		WebAssembly Environment
CWE-121: Stack-based Buffer Overflow	Can detect buffer overflow Fixed stack memory addr		Cannot detect buffer overflow Fixed stack memory addr
CWE-122: Heap-based Buffer Overflow	Can detect buffer overflow Randomized heap memory addr		Cannot detect buffer overflow Fixed heap memory addr
CWE-476: NULL Pointer Dereference	C	C++	Abort
	Segmentation fault	Undefined behavior	



Detailed Findings

CWE-121: Stack-based Buffer Overflow in Rust

	Native Environment	WebAssembly Environment
Testing Compiler	Rustc: official compiler	Wasm-pack
Stack memory address	Randomized	Fixed
Detect Stack-based Buffer Overflow	Yes	Turn into RuntimeError

1. Rust in native environment has randomized stack memory address, while in webassembly have fixed ones in default.
2. The stack-based buffer overflow will cause RuntimeError in webassembly.

Detailed Findings

What is Wasm-pack?



wasm-pack

1. The most common tool to build and work with rust-generated WebAssembly interoped with JavaScript, in the browser or with Node.js.
2. Help to build rust-generated WebAssembly packages that users could publish to the npm registry
3. Ref: <https://github.com/rustwasm/wasm-pack>

Detailed Findings

CWE-121: Stack-based Buffer Overflow in Rust

arg: a_20bytes_1234567890

Execute:

```
let mut buf1 = [0u8; 16];  
let buf2 = [0u8; 16];  
buf1[..8].copy_from_slice(b"password");
```

Stack Data 1: "password", Memory Address: 0x16b42afc0

Stack Data 2: "", Memory Address: 0x16b42afd0

Execute:

```
buf1[..args[1].len()].copy_from_slice(args[1].as_bytes());
```

thread 'main' panicked at stackRust.rs:22:9:
range end index 20 out of range for slice of length 16
note: run with `RUST\_BACKTRACE=1` environment variable
to display a backtrace

Native Environment

arg: a_20bytes_1234567890

```
✖ ▶ Uncaught (in promise) RuntimeError: unreachable  
at stack\_rust\_wasm\_bg.wasm:0x4121  
at stack\_rust\_wasm\_bg.wasm:0x492c  
at stack\_rust\_wasm\_bg.wasm:0x4539  
at stack\_rust\_wasm\_bg.wasm:0x190a  
at stack_test (stack\_rust\_wasm.js:143:14)  
at run (index.html:16:26)
```

arg: a_16bytes_123456

Execute:

```
let mut buf1 = [0u8; 16];  
let buf2 = [0u8; 16];  
buf1[..8].copy_from_slice(b"password");
```

Stack Data: "password", Memory Address: 0xffff98

Stack Data: "", Memory Address: 0xffffa8

Execute:

```
buf1[..arg.len()].copy_from_slice(arg.as_bytes());
```

Stack Data: "a_16bytes_123456", Memory Address: 0xffff98

Stack Data: "", Memory Address: 0xffffa8

WebAssembly Environment



Detailed Findings

CWE-122: Heap-based Buffer Overflow in Rust

	Native Environment	WebAssembly Environment
Testing Compiler	Rustc: official compiler	Wasm-pack
Stack memory address	Randomized	Fixed
Detect Heap-based Buffer Overflow	Yes	Turn into RuntimeError

1. Rust in native environment has randomized heap memory address, while in webassembly have fixed ones in default.
2. The heap-based buffer overflow will cause RuntimeError in webassembly.

Detailed Findings

CWE-122: Heap-based Buffer Overflow in Rust

```
argv[1]: a_20bytes_1234567890
```

Execute:

```
let mut buf1 = vec![0u8; 16];  
let buf2 = vec![0u8; 16];  
buf1[..8].copy_from_slice(b"password");
```

Heap Data 1: "password", Memory Address: 0x15b0041c0

Heap Data 2: "", Memory Address: 0x15b0041d0

Execute:

```
buf1[..args[1].len()].copy_from_slice(args[1].as_bytes());
```

thread 'main' panicked at heapRust.rs:27:9:
range end index 20 out of range for slice of length 16
note: run with `RUST\_BACKTRACE=1` environment variable
to display a backtrace

Native Environment

```
arg: a_20bytes_1234567890
```

```
✖ Uncaught (in promise) RuntimeError: unreachable  
  at heap_rust_wasm_bg.wasm:0x4126  
  at heap_rust_wasm_bg.wasm:0x49d5  
  at heap_rust_wasm_bg.wasm:0x453e  
  at heap_rust_wasm_bg.wasm:0x190f  
  at heap_test (heap_rust_wasm.js:143:14)  
  at run (index.html:16:26)
```

```
arg: a_16bytes_123456
```

Execute:

```
let mut buf1 = vec![0u8; 16];  
let buf2 = vec![0u8; 16];  
buf1[..8].copy_from_slice(b"password");
```

Stack Data: "password", Memory Address: 0x110020

Stack Data: "", Memory Address: 0x110038

Execute:

```
buf1[..arg.len()].copy_from_slice(arg.as_bytes());
```

Stack Data: "a_16bytes_123456", Memory Address: 0x110020

Stack Data: "", Memory Address: 0x110038

WebAssembly Environment



Detailed Findings

Unsafe Rust v.s. Safe Rust

	Safe Rust	Unsafe Rust
Pointer Types	Reference Pointer	Raw Pointer
Code Example	<code>let ref_pointer: &mut i32 = value;</code>	<code>let mut raw_pointer: *mut i32 = value;</code>
Property	Borrow a value without owning it	Directly manipulate memory addresses Bypassing Rust's borrowing rules

Unsafe Rust exists because, by nature, static analysis is conservative.
And Rust needs to allow user to do low-level systems programming, such as directly interacting with the operating system or even writing your own operating system.



Detailed Findings

Unsafe Rust v.s. Safe Rust

- First (and safer) way to use unsafe Rust:
Declare an “unsafe” function, and called in the unsafe block (unsafe { }).
=> It will get error if not called in unsafe block.
- Second way to use unsafe Rust:
Declare a safe function but have unsafe code inside.
=> It will NOT get error if not called in unsafe block.
=> May use unsafe Rust in safe functions!

```
unsafe fn unsafe_rust_function(value: *mut i32) {  
    let mut raw_pointer: *mut i32 = value;  
    raw_pointer = std::ptr::null_mut();  
}  
  
fn main() {  
    let mut value = 10;  
    unsafe {  
        unsafe_rust_function(&mut value as *mut i32);  
    }  
}
```

```
fn unsafe_rust_function(value: *mut i32) {  
    unsafe {  
        let mut raw_pointer: *mut i32 = value;  
        raw_pointer = std::ptr::null_mut();  
    }  
}
```



Detailed Findings

CWE-476: NULL Pointer Dereference in Rust

	Native Environment	WebAssembly Environment
Testing Compiler	Rustc: official compiler	Wasm-pack
Safe Rust Function	Not allowed to change address of pointers	Not allowed to change address of pointers
Unsafe Rust Function	Segmentation fault	Return 0 when meeting null pointer

1. Safe Rust Function: It's not allowed to change address of pointers.
i.e. (X) `ref_pointer = &mut *std::ptr::null_mut()`
2. Unsafe Rust Function: There is segmentation fault in native environment, while in webassembly, it will return 0 instead.



Detailed Findings

CWE-476: NULL Pointer Dereference in Rust

```
safe_rust_function:  
Address of ref_pointer: 0x16d7830bc  
Value of ref_pointer: 10  
  
unsafe_rust_function:  
Address of raw_pointer: 0x16d7830bc  
Value of raw_pointer: 10  
Execute:  
raw_pointer = std::ptr::null_mut();  
Address of raw_pointer: 0x0  
zsh: segmentation fault ./nullPointer
```

Native Environment

```
safe_rust_function  
-----  
Address of ref_pointer: 0xfffa4  
Value of ref_pointer: 10  
  
unsafe_rust_function  
-----  
Address of raw_pointer: 0xff8c  
Value of ref_pointer: 10  
Execute:  
raw_pointer = std::ptr::null_mut();  
Address of raw_pointer: 0x0  
Value of ref_pointer: 0
```

WebAssembly Environment



Detailed Findings

CWE-121: Stack-based Buffer Overflow in Go

	Native Environment	WebAssembly Environment
Testing Compiler	gc	gc with GOOS=js and GOARCH=wasm
Stack memory address	No compiler randomization	Fixed
Detect Stack-based Buffer Overflow	No Issue	No Issue

1. Due to the copy() function in Go any input longer than the space allocated for the slice is truncated preventing Buffer overflow

Detailed Findings

CWE-121: Stack-based Buffer Overflow in Go

```
$ go run stackBufferOverflow.go a_20bytes_1234567890  
argv[1]: a_20bytes_1234567890
```

```
Execute:  
var buf1 [16]byte  
var buf2 [16]byte  
copy(buf1[:], "password")
```

```
Stack Data 1: password, Memory Address: 0xc00000a110  
Stack Data 2: , Memory Address: 0xc00000a120
```

```
Execute:  
copy(buf2[:], os.Args[1])
```

```
Stack Data 1: password, Memory Address: 0xc00000a110  
Stack Data 2: a_20bytes_123456, Memory Address: 0xc00000a120
```

Native Environment

```
input1: a_20bytes_1234567890  
wasm_exec.js:22  
wasm_exec.js:22
```

```
Execute:  
var buf1 [16]byte  
var buf2 [16]byte  
copy(buf1[:], "password")
```

```
Stack Data 1: password, Memory Address: 0x140c050  
wasm_exec.js:22
```

```
Stack Data 2: , Memory Address: 0x140c060  
wasm_exec.js:22
```

```
Execute:  
copy(buf2[:], input1)
```

```
Stack Data 1: password, Memory Address: 0x140c050  
wasm_exec.js:22
```

```
Stack Data 2: a_20bytes_123456, Memory Address: 0x140c060  
wasm_exec.js:22
```

Webassembly Environment



Detailed Findings

CWE-122: Heap-based Buffer Overflow in Go

	Native Environment	WebAssembly Environment
Testing Compiler	gc	gc with GOOS=js and GOARCH=wasm
Stack memory address	No compiler randomization	Fixed
Detect Heap-based Buffer Overflow	No Issue	No Issue

1. Due to the copy() function in Go any input longer than the space allocated for the slice is truncated preventing Buffer overflow

Detailed Findings

CWE-122: Heap-based Buffer Overflow in Go

```
$ go run heapBufferOverflow.go a_20bytes_1234567890 b_20bytes_1234567890  
argv[1]: a_20bytes_1234567890  
argv[2]: b_20bytes_1234567890
```

Execute:

```
var ptr1, ptr2 []byte  
ptr1 = make([]byte, 8)  
ptr2 = make([]byte, 8)
```

```
Heap Data 1: , Memory Address: 0xc0000a0f8  
Heap Data 2: , Memory Address: 0xc0000a130
```

Execute:

```
copy(ptr1, os.Args[1])  
Heap Data 1: a_20byte, Memory Address: 0xc0000a0f8  
Heap Data 2: , Memory Address: 0xc0000a130
```

Execute:

```
copy(ptr2, os.Args[2])  
Heap Data 1: a_20byte, Memory Address: 0xc0000a0f8  
Heap Data 2: b_20byte, Memory Address: 0xc0000a130
```

Native Environment

```
input1: a_20bytes_1234567890 wasm_exec.js:22
```

```
input2: b_20bytes_1234567890 wasm_exec.js:22
```

```
wasw_exec.js:22
```

Execute:

```
var ptr1, ptr2 []byte  
ptr1 = make([]byte, 8)  
ptr2 = make([]byte, 8)
```

```
Heap Data 1: , Memory Address: 0x140c0b0 wasm_exec.js:22
```

```
Heap Data 2: , Memory Address: 0x140c0b8 wasm_exec.js:22
```

```
wasw_exec.js:22
```

Execute:

```
copy(ptr1, input1)
```

```
Heap Data 1: a_20byte, Memory Address: 0x140c0b0 wasm_exec.js:22
```

```
Heap Data 2: , Memory Address: 0x140c0b8 wasm_exec.js:22
```

```
wasw_exec.js:22
```

Execute:

```
copy(ptr2, input2)
```

```
Heap Data 1: a_20byte, Memory Address: 0x140c0b0 wasm_exec.js:22
```

```
Heap Data 2: b_20byte, Memory Address: 0x140c0b8 wasm_exec.js:22
```

Webassembly Environment



Detailed Findings

CWE-476: NULL Pointer Dereference in Go

	Native Environment	WebAssembly Environment
Testing Compiler	gc	gc with GOOS=js and GOARCH=wasm
Stack memory address	No compiler randomization	Fixed
Detect Null Pointer Dereference	Detected and Runtime error	Detected and Runtime error

1. Go contains built in checks that prevent dereferencing a pointer and pointer arithmetic is prohibited
2. Dereferencing a pointer with the value of nil causes runtime panic to handle unexpected conditions disallowing undefined behaviour



Detailed Findings

CWE-476: NULL Pointer Dereference in Go

```
$ go run nullPointerDereference.go v
panic: runtime error: invalid memory address or nil pointer dereference
[signal 0xc0000005 code=0x1 addr=0x0 pc=0xa3af5e]

goroutine 1 [running]:
main.main()
    C:/Users/rbroc/Documents/CS 6V81 INSuRE/UTD Project/CWE476/Go/nullPointerDereference.go:23 +0x5e
exit status 2
```

Native Environment

```
panic: runtime error: invalid memory address or nil pointer dereference      wasm_exec.js:22
[signal 0xb code=0x0 addr=0x0 pc=0x0]                                       wasm_exec.js:22
                                                                              wasm_exec.js:22
goroutine 1 [running]:                                                     wasm_exec.js:22
main.main()                                                                  wasm_exec.js:22
    C:/Users/rbroc/Documents/CS 6V81 INSuRE/UTD Project/CWE476/Go/WebA wasm_exec.js:22
    ssembly/nullPointerDereference.go:20 +0x2
⚠ ▶ exit code: 2                                                            wasm_exec.js:101
```

Webassembly Environment



Detailed Findings: ChatGPT 3.5 vs ChatGPT 4.0

CWE-121: Stack-Based Buffer Overflow

- **Identification of Issue**
 - **ChatGPT 3.5:** Points out vulnerability due to `strcpy` without bounds checking.
 - **ChatGPT 4.0:** Identifies not only the buffer overflow but also uninitialized memory use and missing input validation.
- **Depth of Analysis**
 - **ChatGPT 3.5:** Focuses mainly on buffer overflow and its fix.
 - **ChatGPT 4.0:** Provides a detailed analysis including potential security risks and exploitation.
- **Recommendations**
 - **ChatGPT 3.5:** Advises using `strncpy` to prevent overflows.
 - **ChatGPT 4.0:** Suggests bounds checking, input validation, buffer initialization, and avoiding printing memory addresses.



Detailed Findings: ChatGPT 3.5 vs ChatGPT 4.0

CWE-122: Heap-Based Buffer Overflow

- **Identification of Issue**
 - **ChatGPT 3.5:** Highlights buffer overflow due to insufficient buffer size and `strcpy` usage.
 - **ChatGPT 4.0:** Details buffer overflow, uninitialized memory usage, and missing input validation.
- **Depth of Analysis**
 - **ChatGPT 3.5:** Concentrates on how buffer overflow occurs and its immediate risks.
 - **ChatGPT 4.0:** Discusses broader impacts including potential for exploitation and data exposure.
- **Recommendations**
 - **ChatGPT 3.5:** Recommends `strncpy` for safer string copying.
 - **ChatGPT 4.0:** Encourages checking input lengths, using `calloc`, and proper error handling for memory allocation failures.



Detailed Findings: ChatGPT 3.5 vs ChatGPT 4.0

CWE-476: NULL Pointer Dereference

- **Identification of Issue**
 - **ChatGPT 3.5:** Identifies potential NULL pointer dereference.
 - **ChatGPT 4.0:** Expands on NULL pointer dereference and incorrect memory allocation size.
- **Depth of Analysis**
 - **ChatGPT 3.5:** Focuses on the risk of dereferencing without NULL checks.
 - **ChatGPT 4.0:** Details additional issues like incorrect allocation for strings.
- **Recommendations**
 - **ChatGPT 3.5:** Suggests adding NULL checks before dereferencing pointers.
 - **ChatGPT 4.0:** Recommends adding size for null terminator in allocation and improving error messaging.



Issues / Challenges

- Adapting complex system software for testing with new languages.
- The response time of remote virtual machine is slow than expected.
- Mitigation for Issues:
 - Early pilot testing.
 - Expert consultations.
- Iterative refinement of software test cases.
- Ensuring the accuracy of vulnerability comparisons.
- Team members' inexperience with the new languages.
- Some team members lack a computer science background.
- Mitigation for team challenges:
 - Delegating tasks to group members based on their background.
 - Utilizing online research to learn about the software in use.



Conclusions and Recommendations

- Extensive literature exists on C/C++ memory vulnerabilities and surprisingly there is also a good amount of literature on WebAssembly memory vulnerabilities.
- Literature/Research has shown our desire to test WebAssembly alongside C/C++ compared to “Memory Safe” languages is a quite worthy project.
- Some CVE’s we found are not applicable in WebAssembly (even if they are in C or C++) so this lead us to tailoring our selection to those we could test in both WebAssembly and C languages



Conclusions and Recommendations

- **CWE-121: Stack-based Buffer Overflow:**
In C, WebAssembly environment lacks the detection of stack-based buffer overflow in certain conditions against native environment.
- In Rust, it will prevent program execute buffer overflow in webAssembly environment.
- In Go, buffer overflows are prevented by the safeties built into the language which transfers over to the Webassembly environment.



Conclusions and Recommendations

- CWE-122: Heap-based Buffer Overflow:
In C, WebAssembly environment has the higher risk to get exploitation by heap-based buffer overflow.
- In Rust, it will prevent program execute buffer overflow in webAssembly environment.
- In Go, buffer overflows are prevented by the safeties built into the language which transfers over to the Webassembly environment.



Conclusions and Recommendations

- CWE-476: NULL Pointer Dereference:
C++'s object oriented design allows for undefined behavior to be undetected in certain circumstances.
- In Rust, safe mode prohibit changing address of pointer to prevent NULL pointer dereference.
- In Go, built in checks prevent nil pointers from being dereferenced and pointer arithmetic is prohibited.

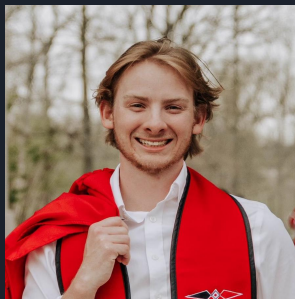


Conclusions and Recommendations

Comparing ChatGPT 3.5 and ChatGPT 4.0 Code Analyses

- **Enhanced Identification**
- **Deeper Analysis**
- **Robust Recommendations**
- **Overall Improvement**
- **ChatGPT can detect the CWE based on the reasons of the surface, like strcpy() usage, but can not give the prediction of different developed environments as the result we got from the real testing program. Especially, it is relatively helpless to give the accurate result regarding new research.**

Investigator Biographies



Joshua McCoy

Education:

- B.S. in Computer Science, Miami University, Oxford, Ohio.
- Graduate Student in Cybersecurity, University of Arizona.

Experience:

- Systems Security Engineer at RTX, Tucson, AZ, focusing on PIT System Cybersecurity and Anti-Tamper for missile systems.
- Software Engineering intern at Microsoft (React.js).
- Data Science intern at The Center for Analytics and Data Science (Data Analytics, R, RStudio).
- Software Developer for The Miami Tribe of Oklahoma (HTML, CSS, JavaScript, SQL).
- Associate Developer Consultant at Cyborg Mobile (React.js).



Chia-Hao Yang

Education:

- B.S. in Electrical Engineering.
- M.S. in Computer Science (in progress) at the University of Texas at Dallas.
- Graduate Certificate in Cyber Defense (NSA CAE).

Experience:

- Top 10% in NSA Codebreaker Challenge 2022.
- 3rd place in TexSAW 2022 CTF competition.



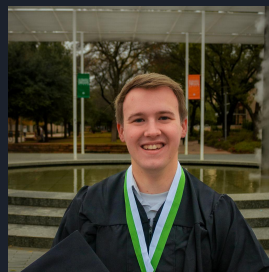
Nate Schultz

Education:

- B.A. in Information Communication Technology and Policy, University of Hawaii.
- M.S. in Management Information Systems, University of Arizona.
- Currently completing a Cybersecurity Master's at the University of Arizona.

Experience:

- 15 years in IT, with the last 3 years as a System Administrator at RTX.



Ryan Brockinton

Education:

- B.S. in Computer Science from the University of Texas at Dallas.
- Pursuing an M.S. in Computer Science with an emphasis in Cybersecurity at the same university.

Experience:

- Participated in Glacier CTF 2023, focusing on reverse engineering, low-level programming, and web exploitation.
- Multiple internships at Texas Instruments, focusing on system automation and web development.



Bibliography

- [1] Daniel Lehmann, Johannes Kinder, Michael Pradel, “Everything Old is New Again: Binary Security of WebAssembly,” in the Proceedings of the 29th USENIX Security Symposium, 2020.
- [2] C. Disselkoen, J. Renner, C. Watt, T. Garfinkel, A. Levy, and D. Stefan, “Position paper: Progressive memory safety for webassembly,” in Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy, ser. HASP '19. New York, NY, USA: Association for Computing Machinery, 2019.
- [3] Quentin Stiévenart, Coen De Roover, “Mohammad Ghafari The Security Risk of Lacking Compiler Protection in WebAssembly,” in International Conference on Software Quality, Reliability and Security, 2021.
- [4] Suhyeon Song, Seonghwan Park, Donghyun Kwon, “metaSafer: A Technique to Detect Heap Metadata Corruption in WebAssembly,” in IEEE Access, 2023.
- [5] Minseo Kim, Hyerean Jang, Young-joo Shin, “Avengers, Assemble! Survey of WebAssembly Security Solutions,” in IEEE International Conference on Cloud Computing, 2022.
- [6] Marius Musch, Christian Wressnegger, Martin Johns, Konrad Rieck, “New Kid on the Web: A Study on the Prevalence of WebAssembly in the Wild,” in International Conference on Detection of intrusions and malware, and vulnerability assessment, 2019.



Appendix A: Acronyms

- **AI:** Artificial Intelligence
- **CWE:** Common Weakness Enumeration
- **CTF:** Capture The Flag (inferred context, not explicitly defined)
- **EMCC:** Emscripten Compiler Frontend (inferred context, not explicitly defined)
- **GDB:** GNU Debugger
- **GCC:** GNU Compiler Collection (inferred context, not explicitly defined)
- **NSA:** National Security Agency
- **NVD:** National Vulnerability Database
- **PIT:** Platform IT
- **Wasm:** WebAssembly



Appendix A: Terms / Definitions

- **CWE (Common Weakness Enumeration):** A categorization of software weaknesses and vulnerabilities.
- **GDB (GNU Debugger):** A debugger offers extensive facilities for tracing and altering the execution of computer programs.
- **NSA (National Security Agency):** A U.S. agency responsible for national security and cryptography.
- **NVD (National Vulnerability Database):** A database that contains security checklist references, security-related software flaws, and misconfigurations.
- **PIT (Platform IT): InforAI (Artificial Intelligence):** Refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. AI can be applied to various sectors including software development for tasks such as vulnerability detection and remediation.
- **CWE (Common Weakness Enumeration):** A list that categorizes common software weaknesses and vulnerabilities. It serves as a standard for identifying, mitigating, and preventing software vulnerabilities.
- **CTF (Capture The Flag):** In cybersecurity, it refers to a competition where participants solve security-related challenges to find a specific piece of data, called the "flag." This term was inferred and not explicitly defined in the provided text.
- **EMCC (Emscripten Compiler Frontend):** A tool used to compile C and C++ code into WebAssembly. It allows developers to run C and C++ code on the web at near-native speed.
- **GDB (GNU Debugger):** A debugger that provides extensive facilities for tracing and altering the execution of computer programs. It's part of the GNU Project and supports various programming languages including C and C++.
- **GCC (GNU Compiler Collection):** A compiler system produced by the GNU Project supporting various programming languages. It's standard for compiling C and C++ code, among others.
- **NSA (National Security Agency):** A U.S. government agency responsible for national security and the protection of government communications and information systems against penetration and network warfare.
- **NVD (National Vulnerability Database):** A database that contains information about security-related software flaws, misconfigurations, product names, and impact metrics. It's managed by the National Institute of Standards and Technology (NIST).
- **PIT (Platform IT):** Refers to information technology solutions and systems that support specific platforms or operations, often in a specialized or embedded context. This term was inferred and not explicitly defined in the provided text.
- **Wasm/WebAssembly:** A binary instruction format for a stack-based virtual machine. WebAssembly is designed as a portable compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications.

Questions?

